

# GameObject et composants

Dans Unity, un **GameObject** est l'entité de base dans une scène. Il peut représenter des personnages, des objets, des lumières, des caméras, et bien plus encore. Cependant, un GameObject en lui-même n'a pas de comportement ou d'apparence spécifique. C'est là que les **composants** entrent en jeu. Il s'agit de modules que vous pouvez ajouter à un GameObject pour lui donner des fonctionnalités spécifiques. Nous avons vu des composants tels que :

- **SpriteRenderer** : Affiche une image 2D (sprite) sur le GameObject.
- **Transform** : Gère la position, la rotation et l'échelle du GameObject dans l'espace 3D.
- **Rigidbody2D** : Permet au GameObject de réagir aux forces physiques, comme la gravité et les collisions.
- **Collider2D** : Définit la forme de collision du GameObject pour détecter les interactions avec d'autres objets.

## 1.1 Propriété/méthodes du GameObject

Voici quelques autres méthodes utiles que vous pouvez utiliser avec les GameObjects :

- `name` : Permet d'obtenir ou de définir le nom du GameObject. C'est le nom que vous voyez dans le haut de l'Inspector.
- `tag` : Permet d'obtenir ou de définir le tag du GameObject. Les tags sont utilisés pour catégoriser les GameObjects et faciliter leur identification. Ex : le tag "Player" pour le personnage principal et "Enemy" pour tous les ennemis. C'est utile pour les collisions et les interactions.
- `transform` : Permet d'accéder au composant Transform du GameObject, qui gère sa position, rotation et échelle dans la scène.
- `parent` : Permet d'obtenir ou de définir le parent du GameObject dans la hiérarchie des objets. Un GameObject peut être un enfant d'un autre GameObject, ce qui affecte sa position et son comportement.
- `SetActive(bool)` : Active ou désactive le GameObject. Lorsqu'un GameObject est désactivé, il n'est plus mis à jour ni rendu dans la scène.
- `GetComponent<T>()` : Permet d'obtenir une référence à un composant spécifique attaché au GameObject. Par exemple, `GetComponent<Rigidbody2D>()` retourne le composant Rigidbody2D du GameObject.
- `Find(string name)` : Méthode statique qui permet de trouver un GameObject dans la scène par son nom.
- `FindWithTag(string tag)` : Méthode statique qui permet de trouver un GameObject dans la scène par son tag.

**Une méthode statique** signifie que vous l'appellez directement sur la classe `GameObject` elle-même, plutôt que sur un objet réel dans la scène. Exemple : `GameObject.Find("Player")`.

## 1.2 Rechercher un GameObject dans la scène

Parfois, vous aurez besoin de trouver un GameObject spécifique dans votre scène par son nom. Unity fournit une méthode pratique pour cela : `GameObject.Find("NomDuGameObject")`. Cette méthode retourne une référence au GameObject

correspondant au nom spécifié ou un chemin dans la hiérarchie.

```
GameObject player = GameObject.Find("Player");
if (player != null)
{
    // Le GameObject "Player" a été trouvé, vous pouvez maintenant interagir avec lui
}
else
{
    // Le GameObject "Player" n'a pas été trouvé
}
```

```
// Cherche un GO nommé Lune qui est fils d'un appelé Terre.
GameObject lune = GameObject.Find("Terre/Lune");
```

Vous pourriez également utiliser `GameObject.FindWithTag("TagDuGameObject")` pour trouver un `GameObject` par son tag, ce qui est souvent plus efficace si vous avez plusieurs objets avec le même nom. Pour le moment, nous n'avons pas vu les tableaux donc nous rechercherons un seul `GameObject` avec ce tag mais il est possible d'en récupérer plusieurs avec `GameObject.FindGameObjectsWithTag("TagDuGameObject")`, qui retourne un tableau de `GameObjects`. (il y a un "s" à `GameObjects`).

```
GameObject enemy = GameObject.FindWithTag("Enemy");
if (enemy != null)
{
    // Le GameObject avec le tag "Enemy" a été trouvé
}
else
{
    // Aucun GameObject avec le tag "Enemy" n'a été trouvé
}
```

### 1.3 Accéder aux composants par script

Pour interagir avec les composants d'un `GameObject` via un script, vous pouvez utiliser la méthode `GetComponent<T>()`, où `T` est le type du composant que vous souhaitez accéder. C'est une bonne pratique de stocker une référence au composant dans une variable pour éviter d'appeler `GetComponent` plusieurs fois, ce qui peut être coûteux en termes de performance. On place souvent cette initialisation dans la méthode `Start()` ou `Awake()` au début du cycle de vie du script.

Ensuite, vous pouvez utiliser cette référence pour accéder aux propriétés et méthodes du composant.

Par exemple, pour accéder au composant `Rigidbody2D` d'un `GameObject`, vous pouvez faire comme suit :

```
Rigidbody2D rb;
SpriteRenderer sr;
```

```
void Start()
{
    rb = GetComponent<Rigidbody2D>();
    sr = GetComponent<SpriteRenderer>();
}
```

Une fois que vous avez la référence au composant, vous pouvez l'utiliser pour manipuler le GameObject. Par exemple, pour appliquer une force au Rigidbody2D ou changer la couleur du SpriteRenderer :

```
void Update()
{
    // Appliquer une force vers le haut
    if (Keyboard.current.spaceKey.wasPressedThisFrame)
    {
        rb.AddForce(new Vector2(0, 300f));
    }

    // Changer la couleur du sprite en rouge
    if (Keyboard.current.rKey.wasPressedThisFrame)
    {
        sr.color = Color.red;
    }
}
```

Avec cette approche, vous pouvez facilement interagir avec les composants de vos GameObjects et créer des comportements dynamiques dans votre jeu.

## 1.4 Envoyer et diffuser des messages

Une option pour communiquer entre GameObjects est d'envoyer des messages entre GameObjects au moment du runtime. Cette option est bien pratique et flexible, mais elle a un coût d'exécution élevé et est un peu fragile parce qu'elle dépend de donner le nom d'une fonction comme un argument `string`. En général, n'utilisez pas cette technique dans `Update()`, par exemple.

La fonction **BroadcastMessage** permet d'appeler une méthode nommée sans préciser dans quel composant elle est implémentée. Vous pouvez l'utiliser pour appeler une méthode nommée sur chaque MonoBehaviour d'un GameObject ou de tous ses enfants. Vous pouvez également choisir de forcer l'existence d'au moins un récepteur (sinon, une erreur est générée) avec l'argument **SendMessageOptions**.

La fonction **SendMessage** est plus précise : la méthode nommée est appelée uniquement sur le GameObject lui-même, et non sur ses enfants.

La méthode **SendMessageUpwards** appelle la méthode nommée sur le GameObject et toute sa chaîne des parents dans la hiérarchie.

### 1.4.1 Exemple de communication avec messages

```
// Composant attaché à "GO 1"
using UnityEngine;

public class Test01 : MonoBehaviour
{
    void Start()
    {
        // Cherche une référence à un GO actif dans la scène avec le tag ObstacleUnique.
        GameObject cible = GameObject.FindGameObjectWithTag("ObstacleUnique");
        if(cible != null)
        {
            Debug.Log("Message envoyé par " + gameObject.name);
            // Ce message est envoyé a ce GameObject et tous ces enfants.
            // Si un des composants a la méthode RecevoirMessage, elle va être appelée
            cible.BroadcastMessage("RecevoirMessage", SendMessageOptions.DontRequireReceiver);

            // Ce message est envoyé seulement a ce GameObject.
            // Si un de ses composants a la méthode RecevoirMessage, elle va être appelée
            cible.SendMessage("RecevoirMessage", SendMessageOptions.DontRequireReceiver);
        }
    }
}
```

```
// Composant attaché à "GO 2"
using UnityEngine;

public class Test02 : MonoBehaviour
{
    void RecevoirMessage()
    {
        Debug.Log($"MSG reçu au {gameObject.name}");
    }
}
```

[Documentation officielle de Unity sur les GameObjects et les composants](#) [Documentation officielle de Unity sur les Composants](#) [Documentation officielle de Unity sur GetComponent](#) [Documentation officielle de Unity sur les tags](#)